

Datatypes Provide

Validation

Efficient storage

Faster searching

Better indexing

Arithmetic operations

Sorting

Comparisons

Query optimization

SQL requires every column to have a datatype

Every datatype has:

- storage size
- binary representation
- comparison rules
- indexing behavior
- arithmetic rules

Suppose Aadya's shopping website stores

Age as TEXT

Problems

- More storage
- Slower comparisons
- Incorrect sorting
- Larger indexes
- Poor performance

Numeric Types

Store numbers.

Examples

INT

BIGINT

DECIMAL

FLOAT

DOUBLE

Character Types

Store text.

Examples

CHAR

VARCHAR

TEXT

Date & Time Types	Store dates and times. Examples DATE TIME DATETIME TIMESTAMP
Binary Types	Store raw binary data. Examples BINARY VARBINARY BLOB
Boolean Types	Store true/false values. Examples BOOLEAN BIT
JSON Type	Stores structured JSON documents. Example <pre>{ "theme": "dark", "language": "English" }</pre>

CHAR(10)

Suppose you store

Aadya

Only 5 characters.

MySQL still reserves space for 10 characters (padding the remaining positions internally).

Storage

```
|A|a|d|y|a| | | | | |
```

Advantages

- Faster for values with the same length.
- Predictable storage.

Disadvantages

- Wastes space for shorter values.

Variable-Length

Example

`VARCHAR(10)`

Store

`Aadya`

Only the required characters (plus a small amount of length metadata) are stored.

Advantages

- Saves storage.
- Better for varying-length text.

Disadvantages

- Slightly more overhead than fixed-length storage.

Why Not Store Everything as TEXT?

One of the best interview questions.

Imagine

```
CREATE TABLE Student(  
  ID TEXT,  
  Name TEXT,  
  CGPA TEXT,  
  DOB TEXT  
);
```

Everything works...

until it doesn't.

Sorting

Text sorting:

100

20

5

becomes

100

20

5

Arithmetic

Can you calculate

Salary + Bonus

Memory

Suppose

Age

21

Stored as INT

≈ 4 bytes.

Stored as TEXT

Requires character storage plus additional overhead.

Index Performance

Indexes on integers are generally:

- smaller
- faster
- easier to compare

than indexes on long text values.

Age

stored as

TEXT

Someone inserts

Twenty One

The database accepts it.

Now your data is inconsistent.

Authored by : Arpit Raj , Lnmiit jaipur

Chapter 8 — Numeric Data Types

Topics

Integer Types

TINYINT

SMALLINT

MEDIUMINT

INT

BIGINT

Decimal Types

FLOAT

DOUBLE

DECIMAL

Learn for each datatype

Storage Size

Range

Precision

Performance

Advantages

Disadvantages

Real-world Uses

Interview Questions

FLOAT vs DOUBLE

FLOAT vs DECIMAL

INT vs BIGINT

TINYINT

The smallest integer type in MySQL.

Storage

1 byte (8 bits)

SMALLINT

Storage

2 bytes

Range

Signed:

MEDIUMINT (MySQL-specific)

One datatype many developers outside MySQL don't know.

Storage

3 bytes

NT

The most commonly used integer datatype.

Storage

4 bytes

BIGINT

Largest integer datatype.

Storage

8 bytes

Range

Signed:

Decimal Types

8.7 FLOAT

Stores approximate decimal numbers.

Storage

Typically 4 bytes

Precision

About 7 significant digits.

FLOAT follows the IEEE 754 floating-point standard.

Values are stored approximately in binary.

That means:

0.1

cannot always be represented exactly.

Example

0.1 + 0.2

may internally become something like:

0.3000000119

This is a property of binary floating-point arithmetic

Disadvantages

Rounding errors.

DOUBLE

A larger floating-point datatype.

Storage

8 bytes

Precision

About 15–16 significant digits.

Disadvantages

Still approximate.

DECIMAL

The most important datatype for money.

Stores exact decimal values.

Syntax

```
Price DECIMAL(10,2)
```

Meaning

10

total digits.

2

digits after the decimal point.

Examples

Allowed:

12345.67

99999999.99

Internal Working

Unlike FLOAT,

DECIMAL stores the number in a way that preserves the exact decimal value.

So

0.1

is stored exactly.

No binary rounding error.

Disadvantages

- Slightly slower arithmetic than FLOAT/DOUBLE.
- Storage depends on precision and scale.

Character Types (Implied)

CHAR

CHAR stores fixed-length strings.

Syntax

```
CountryCode CHAR(2)
```

Suppose we insert

IN

Exactly 2 characters.

Maximum Length

Maximum:

255 characters

VARCHAR

Most commonly used string datatype.

Stores variable-length strings.

Syntax

```
Name VARCHAR(100)
```

Now

Insert

Aadya

Only 5 characters are stored (plus a small amount of length metadata).

Maximum Length

The theoretical maximum is 65,535 bytes per row

TEXT

TEXT stores large strings.

Unlike VARCHAR, it's designed for long textual content

Large values are stored efficiently, and InnoDB may store very long TEXT values outside the main row with a pointer kept in the row.

Advantages

- Handles large text.
- No need to guess an upper limit like VARCHAR(5000).

Disadvantages

- Less suitable for indexing than shorter string types.
- Often slower than VARCHAR for small strings.

Type	Maximum Size
TINYTEXT	255 bytes
TEXT	65,535 bytes (~64 KB)
MEDIUMTEXT	~16 MB
LONGTEXT	~4 GB

Date & Time Types

Categories

Datatype	Stores
DATE	Date only
TIME	Time only
DATETIME	Date + Time
TIMESTAMP	Date + Time + Time Zone Conversion
YEAR	Year only

DATE

Stores only the calendar date.

Syntax

`Birthday DATE`

Format

`YYYY-MM-DD`

Example

`2005-07-15`

Storage

3 bytes

Example

```
CREATE TABLE Student(  
    Name VARCHAR(100),  
    DOB DATE  
);
```

Insert

```
INSERT INTO Student VALUES  
( 'Aadya', '2006-08-20' );
```

Advantages

- Small storage.
- Efficient comparisons.
- Supports date calculations.

Disadvantages

- Cannot store time.

10.2 TIME

Stores only time.

Format

HH:MM:S

Storage

3 bytes

10.3 DATETIME

Stores both

- Date
- Time

Format

YYYY-MM-DD HH:MM:SS

Storage

5 bytes

TIMESTAMP

One of the most misunderstood datatypes.

Stores

- Date
- Time

but behaves differently from DATETIME.

Format

YYYY-MM-DD HH:MM:SS

Storage

4 bytes

Internally,

MySQL converts it to UTC for storage.

When you retrieve it,

MySQL converts it back to the session's current time zone.

So the stored value is time-zone aware.

Advantages

- Smaller storage.
- Automatic time zone conversion.
- Excellent for recording real-world instants.

Disadvantages

- Limited range (1970–2038). (overflow issues)

YEAR

Stores only a year.

Storage

1 byte

Range

1901

to

2155

Automatic Timestamp

One of MySQL's most useful features.

Suppose

Every time a row changes,

you want to update

LastModified

Automatically.

Example

```
CREATE TABLE Employee(  
  EmployeeID INT AUTO_INCREMENT PRIMARY KEY,  
  Name VARCHAR(100),  
  LastUpdated TIMESTAMP  
    DEFAULT CURRENT_TIMESTAMP  
    ON UPDATE CURRENT_TIMESTAMP  
);
```

Boolean Types

`BOOLEAN`

and

`BOOL`

are simply aliases for

`TINYINT(1)`

```
INSERT INTO Users VALUES (TRUE);  
INSERT INTO Users VALUES (FALSE);
```

Actually stored as

`1`

`0`

Retrieve

```
SELECT IsActive FROM Users;
```

Output

`1`

`0`

Difference between

`BOOLEAN`

and

`BOOL`

Answer

None.

BIT

Example

`Permissions BIT(4)`

Can store

`1010`

`1111`

`0001`

A BIT(1) column uses 1 bit logically, though storage is allocated in bytes internally

Suppose

`BIT(8)`

Value

`10110110`

This is stored as actual bits.

Not characters.

Not integers.

Example

```
CREATE TABLE Flags(  
    Permission BIT(8)  
);
```

Insert

```
INSERT INTO Flags VALUES (b'10101010');
```

Notice

Binary literal

`b'10101010'`

Retrieve

```
SELECT BIN(PERMISSION) FROM Flags;
```

Output

`10101010`

Why BIT Exists?

Suppose

A file has permissions

- Read
- Write
- Execute
- Delete

Instead of four separate Boolean columns

- Read
- Write
- Execute
- Delete

Store

`1110`

One bit per permission.

This is memory efficient and useful for bitwise operations.

Not suitable for historical or far-future dates.